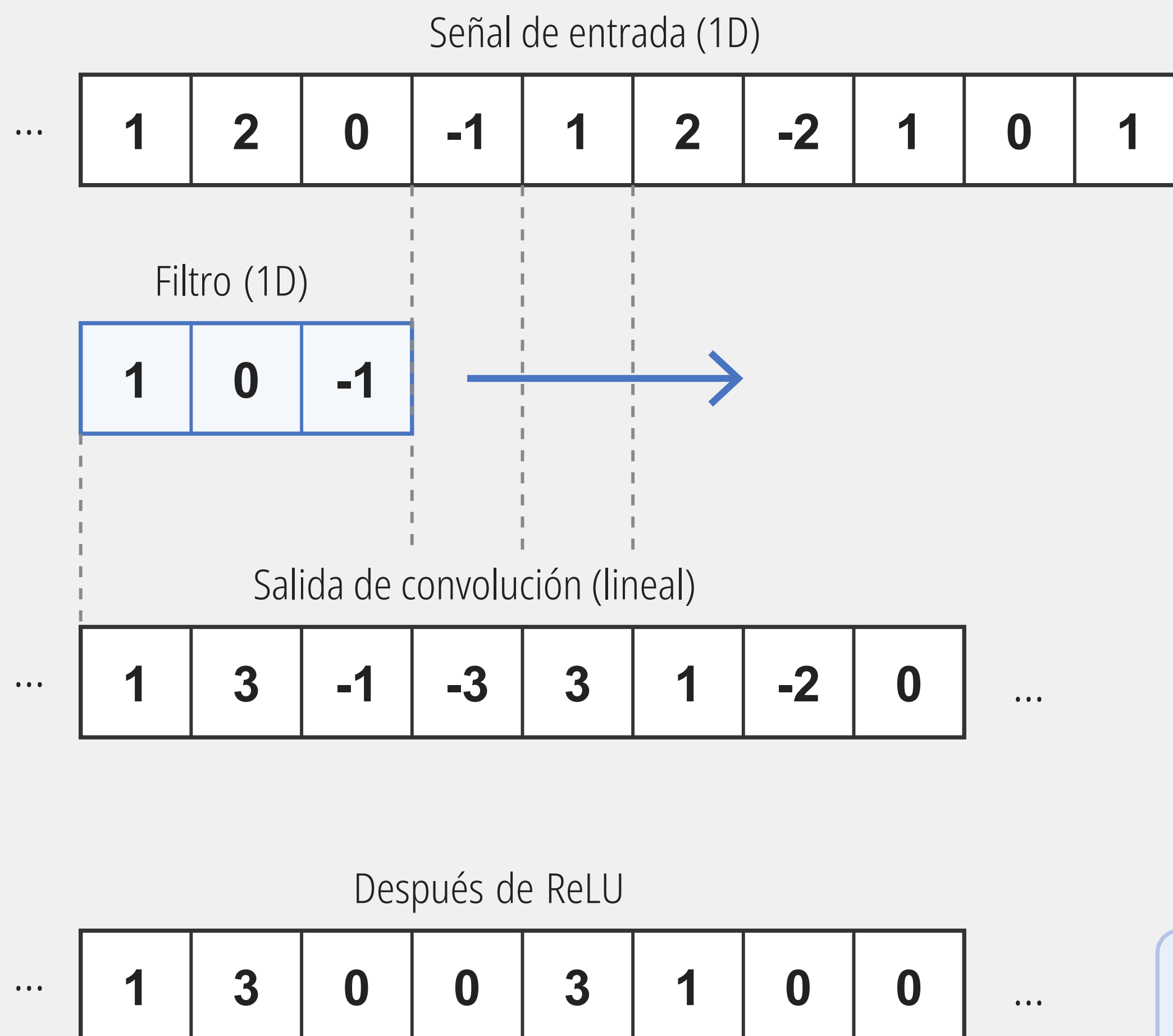


Operación de Convolución (Filtro 1D)

El filtro actúa como un detector de patrones. Responde con valores más altos (como el 3) cuando el segmento de la señal de entrada coincide fuertemente con la “forma” o característica que el filtro está buscando.



- Un pequeño filtro se desliza a lo largo de una señal 1D
- Responde cuando el patrón local coincide
- Después de la convolución, a menudo aplicamos ReLU
- ReLU mantiene las activaciones positivas y pone a cero las negativas

$$\text{ReLU}(x) = \max(0, x)$$

Operación de Convolución (Filtro 2D)

Se desliza tanto horizontal como verticalmente sobre la matriz de entrada de 5 x 5. En cada posición, calcula un producto punto entre los valores del filtro y el parche local de la imagen que está observando (lo que el diagrama llama el campo receptivo).

Imagen de entrada
(5 × 5)

1	0	2	-1	0
0	1	0	1	-1
2	0	1	0	2
-1	1	0	1	0
0	-1	2	0	1

Filtro (3 × 3)

1	0	-1
0	1	0
-1	0	1

Mapa de características
de salida (3 × 3)

-1	1	4
1	1	1
4	1	-1

- Un filtro de $K_h \times K_w$ escanea una imagen 2D
- En cada posición, calcula un producto punto con el parche local
- Deslizarlo sobre toda la imagen produce un mapa de características 2D
- Las regiones locales vistas por el filtro son campos receptivos

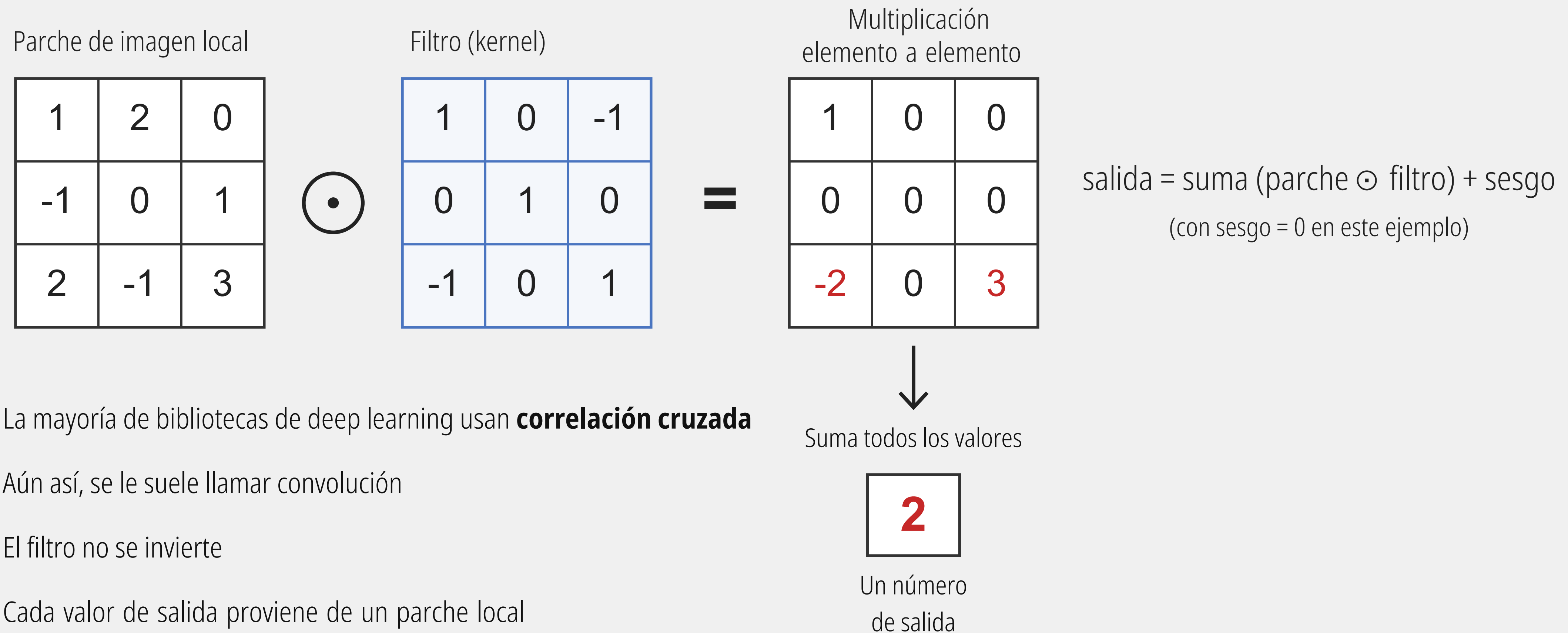


Entrada 5 × 5, filtro 3 × 3, paso 1, relleno 0 → Salida 3 × 3

El campo receptivo de una sola celda en el mapa de salida (por ejemplo, ese -1) corresponde exactamente a los 3 x 3 píxeles específicos de la imagen de entrada original que se utilizaron para calcularlo. A medida que la red neuronal se hace más profunda, los campos receptivos abarcan porciones cada vez más amplias de la imagen original.

Operación de Producto Punto Local (Convolución 2D)

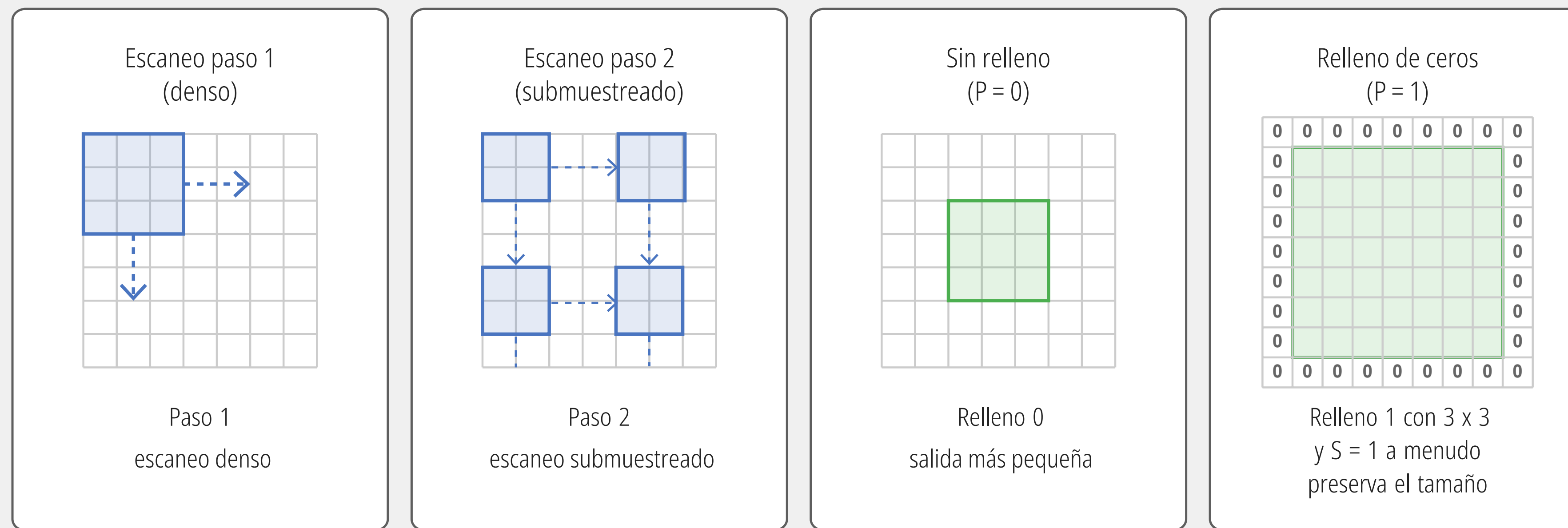
A diferencia del producto matricial tradicional (donde las filas se multiplican por las columnas), aquí se realiza una multiplicación elemento a elemento (conocido como producto de Hadamard).



NOTA: En la práctica del deep learning, omitir la inversión no afecta el rendimiento del modelo. Dado que los valores del filtro se aprenden desde cero durante el entrenamiento (a través del descenso de gradiente), la red neuronal simplemente aprende directamente los pesos "ya invertidos" óptimos para la tarea de clasificación sin gastar ciclos de CPU/GPU en rotar la matriz manualmente en cada pasada.

Paso (stride) y relleno (padding)

Se desglosan visualmente los hiperparámetros espaciales fundamentales de las operaciones en Redes Neuronales Convolucionales (CNNs).



- Paso S = qué tan lejos se mueve el filtro en cada paso
- Un paso mayor da una salida más pequeña
- El relleno P añade valores en el borde, usualmente ceros
- El relleno ayuda a preservar información del borde y el tamaño espacial

$$H_{out} = \left\lfloor \frac{H + 2P - K_h}{S} \right\rfloor + 1$$

$$W_{out} = \left\lfloor \frac{W + 2P - K_w}{S} \right\rfloor + 1$$

El stride o paso define cuántos píxeles se desplaza el filtro (kernel) sobre la matriz de entrada en cada iteración.

Paso 1 (Escaneo Denso): Al configurar un S = 1, el filtro se mueve de un píxel en un píxel. Esto genera una extracción de características muy densa y detallada, pero produce mapas de características más grandes y es computacionalmente más costoso.

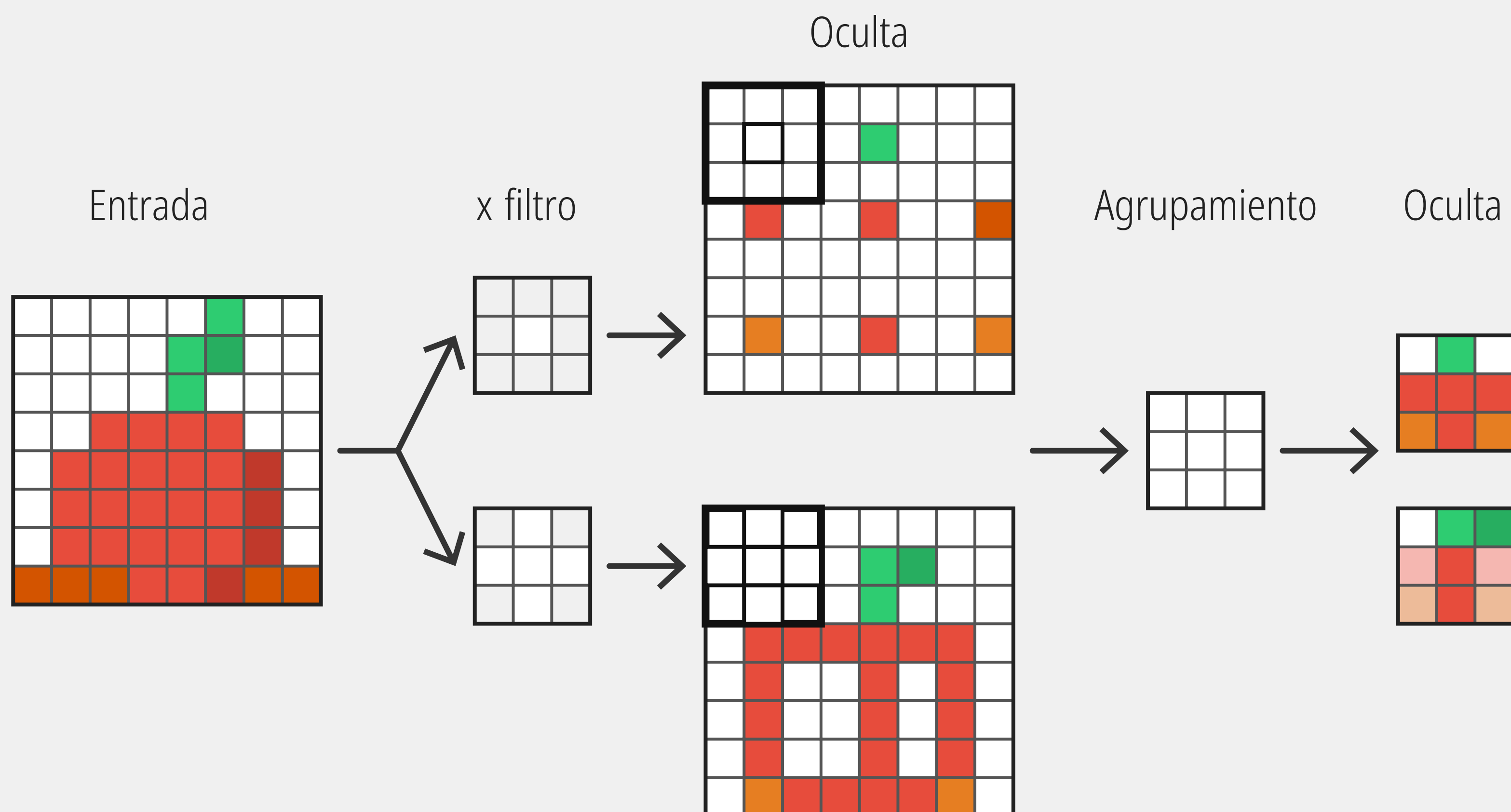
Paso 2 (Escaneo Submuestreado): Al configurar un S = 2, el filtro salta de dos en dos píxeles. El resultado es un submuestreo espacial (downsampling), lo que reduce la resolución de la salida, perdiendo algo de información granular pero reduciendo drásticamente la carga computacional en capas profundas.

El padding consiste en añadir un borde artificial (generalmente ceros) alrededor de la matriz de entrada antes de aplicar la convolución. Esto resuelve dos problemas: la pérdida de información en los bordes y la reducción abrupta del tamaño espacial.

La parte inferior derecha del diagrama presenta las fórmulas matemáticas exactas para calcular la altura y el ancho de la matriz de salida tras aplicar la convolución. Se aplica una operación piso (floor), que redondea hacia abajo al entero más cercano en caso de divisiones no exactas.

Redes Neuronales: Filtros paralelos y agrupamiento (pooling)

En la práctica, una capa convolucional no usa un solo filtro, sino decenas o cientos de ellos simultáneamente. Cada filtro inicializa con valores distintos para especializarse en encontrar un patrón particular.



Primera Capa Oculta (Mapas de Características): La aplicación de los dos filtros da como resultado dos “mapas de características” distintos (las dos cuadrículas grandes en el centro). Observa cómo cada filtro extrajo información completamente diferente de la misma manzana:

Mapa superior: Este filtro está especializado en detectar texturas o puntos aislados (retiene manchas muy específicas de la hoja o el cuerpo).

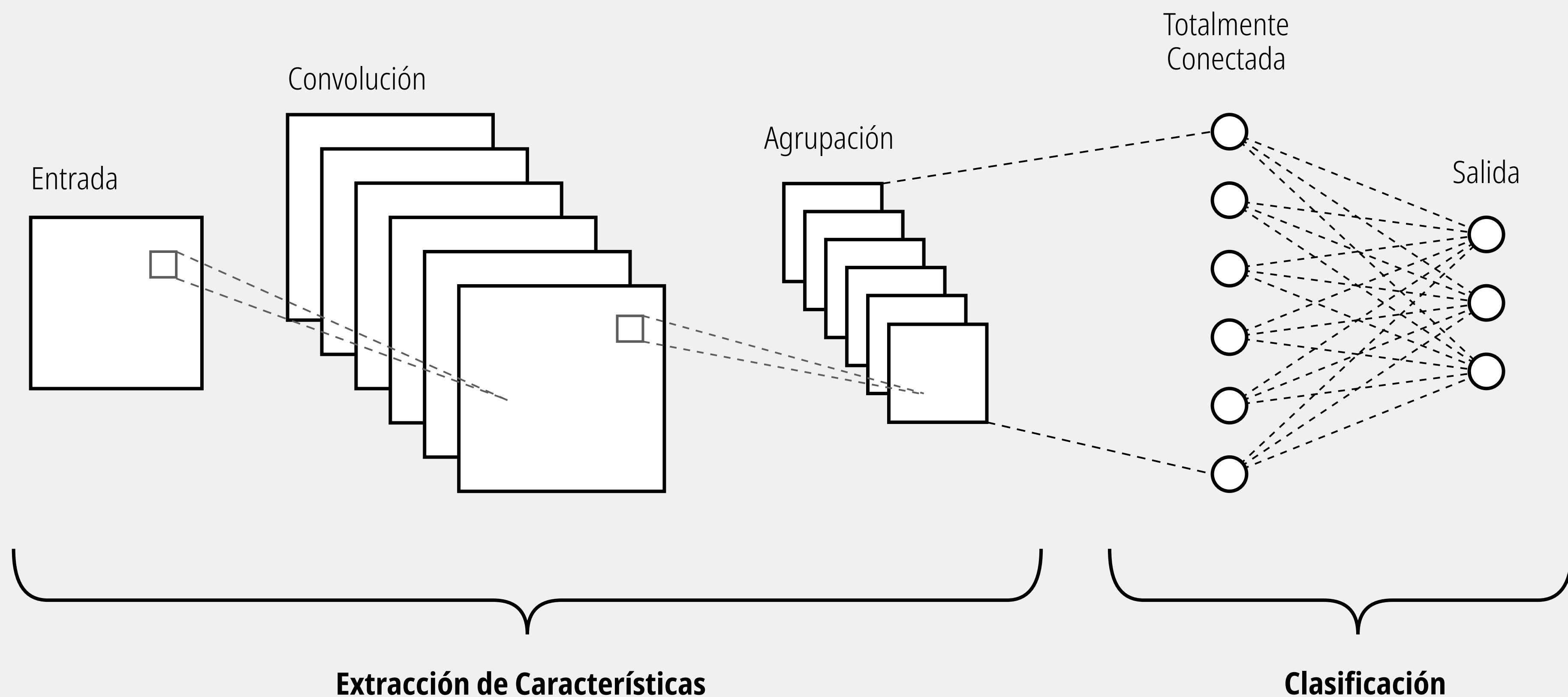
Mapa inferior: Este filtro actúa claramente como un detector de bordes. Observa cómo el mapa resultante es básicamente la silueta o el contorno hueco de la manzana, habiendo descartado todo el color de relleno sólido.

Agrupamiento (Pooling): El recuadro negro con bordes gruesos que se posiciona sobre los mapas de características, junto con el bloque etiquetado como “Agrupamiento”, ilustra la operación de submuestreo (comúnmente Max Pooling o Average Pooling). Consiste en deslizar una nueva ventana sobre los mapas de características recién creados. En lugar de hacer multiplicaciones complejas, simplemente resume la información de esa región local (por ejemplo, si es Max Pooling, solo conserva el píxel con el valor más alto del recuadro negro y descarta el resto).

Salida Final: 1) Reducción de dimensionalidad: El tamaño espacial se ha reducido drásticamente. 2) Invarianza espacial: Aunque la imagen es más pequeña y “pixelada”, retiene la esencia del patrón detectado. La red ya no necesita saber el píxel exacto donde estaba un borde, solo necesita saber que “había un borde en la zona superior derecha”. Esto reduce enormemente el consumo de memoria y la carga computacional para las siguientes capas de la red.

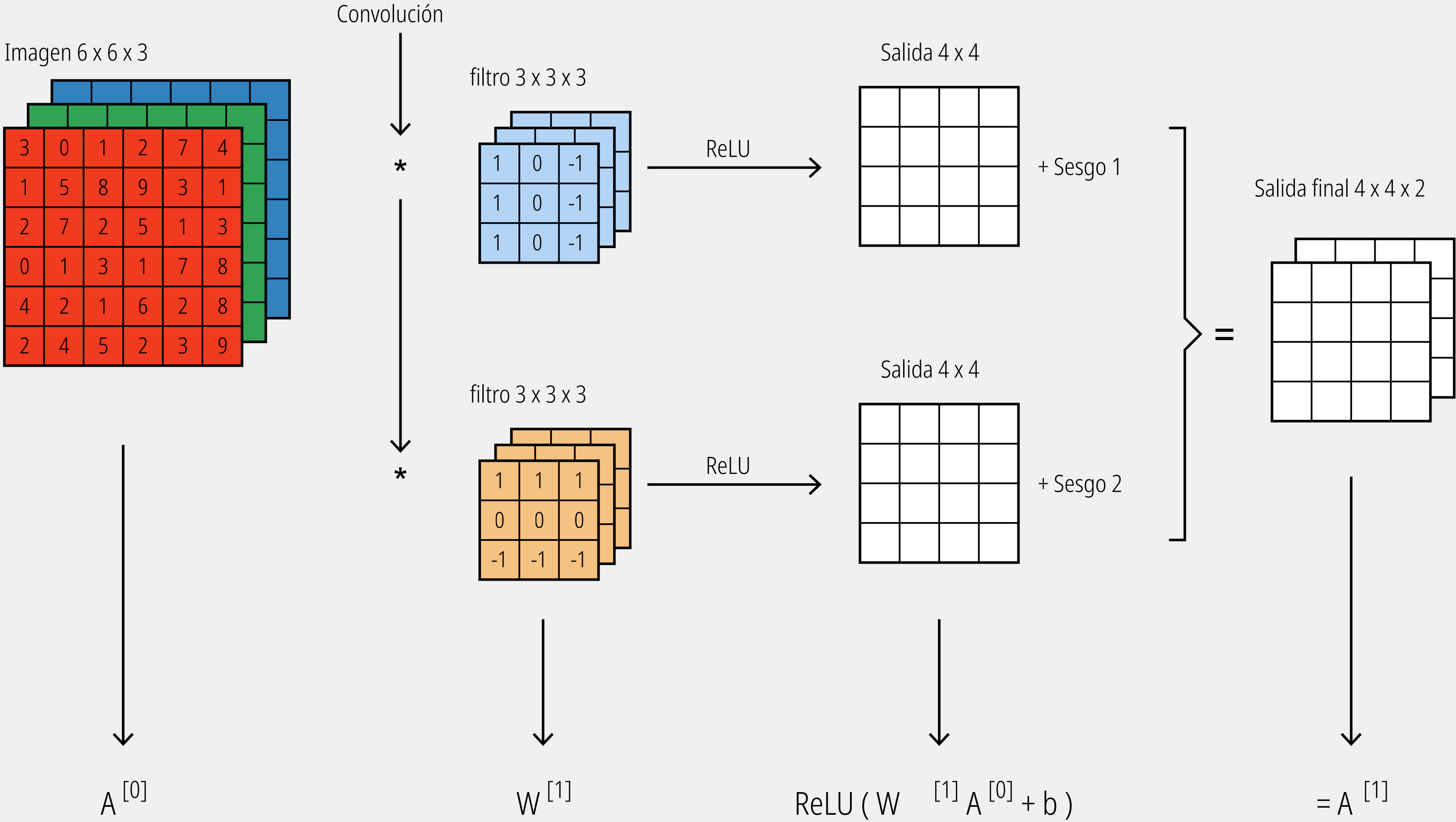
Clasificación con redes neuronales convolutivas

Aunque el diagrama muestra solo una iteración, en arquitecturas profundas reales, este bloque de Convolución + Agrupación se repite múltiples veces. Cada bloque extrae características cada vez más complejas (pasando de simples bordes a texturas, y luego a partes de objetos).



Convolución: Se aplican múltiples filtros simultáneos para crear una "pila" de mapas de características. Las líneas punteadas muestran cómo un pequeño parche de la entrada se mapea a un único píxel en la siguiente capa (el campo receptivo).
 Agrupación (Pooling): Reduce la dimensionalidad espacial de cada mapa de características, reteniendo la información más importante y reduciendo la carga computacional. Dado que la siguiente fase de la red solo entiende datos en forma de vector unidimensional (1D), todas las matrices 2D resultantes de la extracción de características se "desenrollan" en una única y larga columna de valores. Una vez que las características espaciales han sido extraídas y aplanadas, se introducen en una red neuronal tradicional (Perceptrón Multicapa)

Redes neuronales convolutivas (CNN)

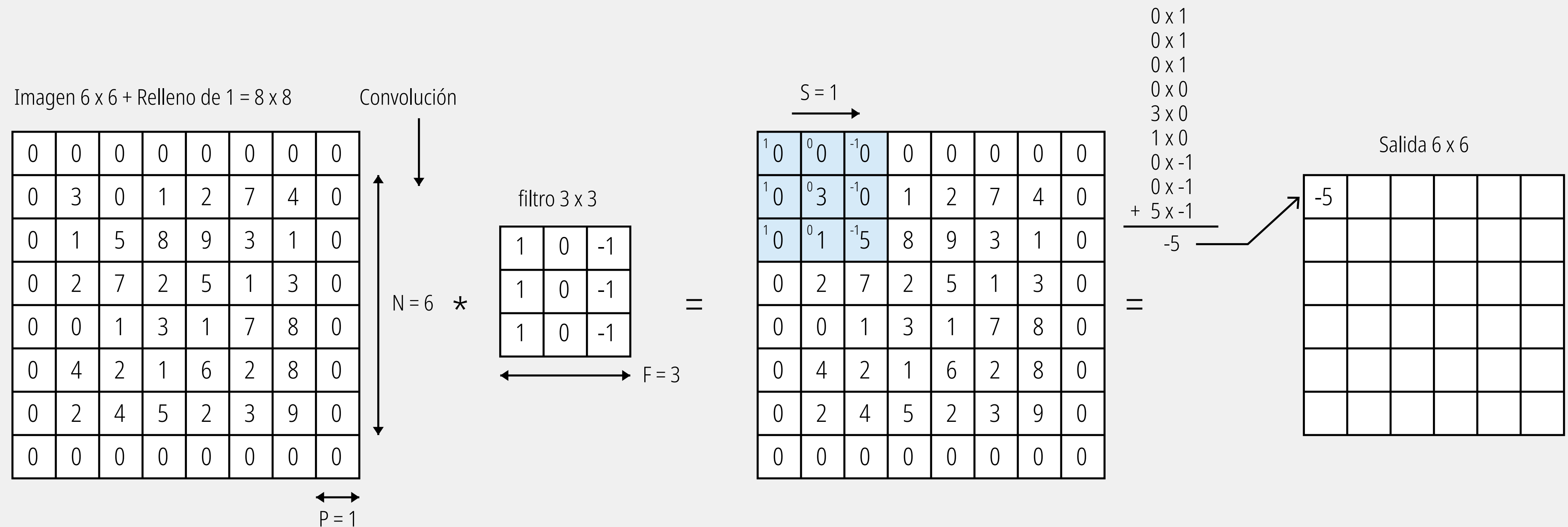


Dimensiones

Entrada	$n_H^{[l-1]} \times n_W^{[l-1]} \times n_c^{[l-1]}$
Activación $a^{[l]}$	$n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$
Pesos	$f^{[l]} \times f^{[l]} \times n_c^{[l-1]} \times n_c^{[l]}$
Sesgo	$n_c^{[l]}$

Donde
 $f^{[l]}$ = tamaño del filtro
 $p^{[l]}$ = relleno
 $s^{[l]}$ = paso
 $n_c^{[l]}$ = número de filtros
 $n_c^{[l-1]}$ = canales de color

Redes neuronales convolutivas (CNN)



El tamaño de salida = $\frac{N - F + 2P}{S} + 1$

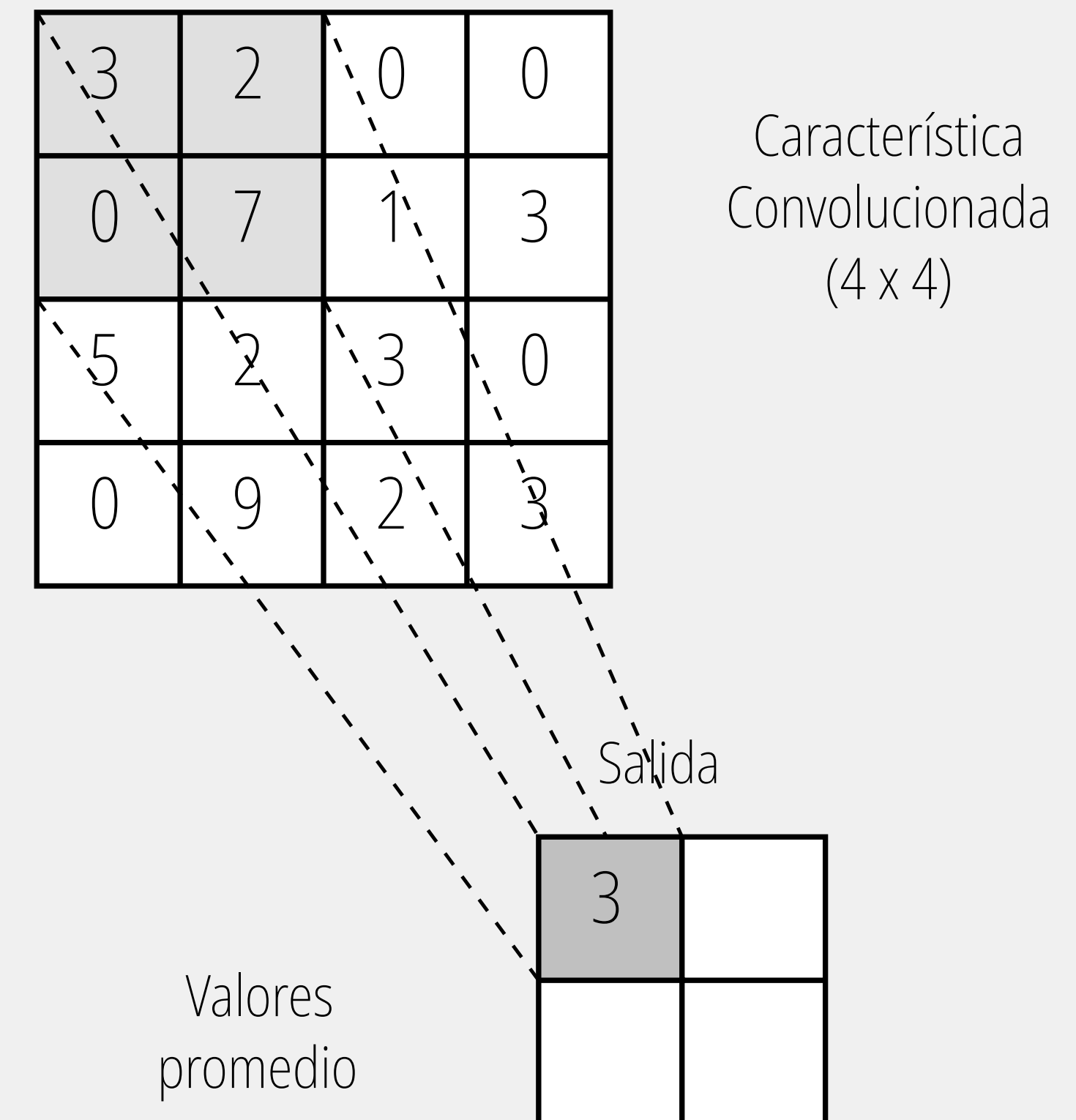
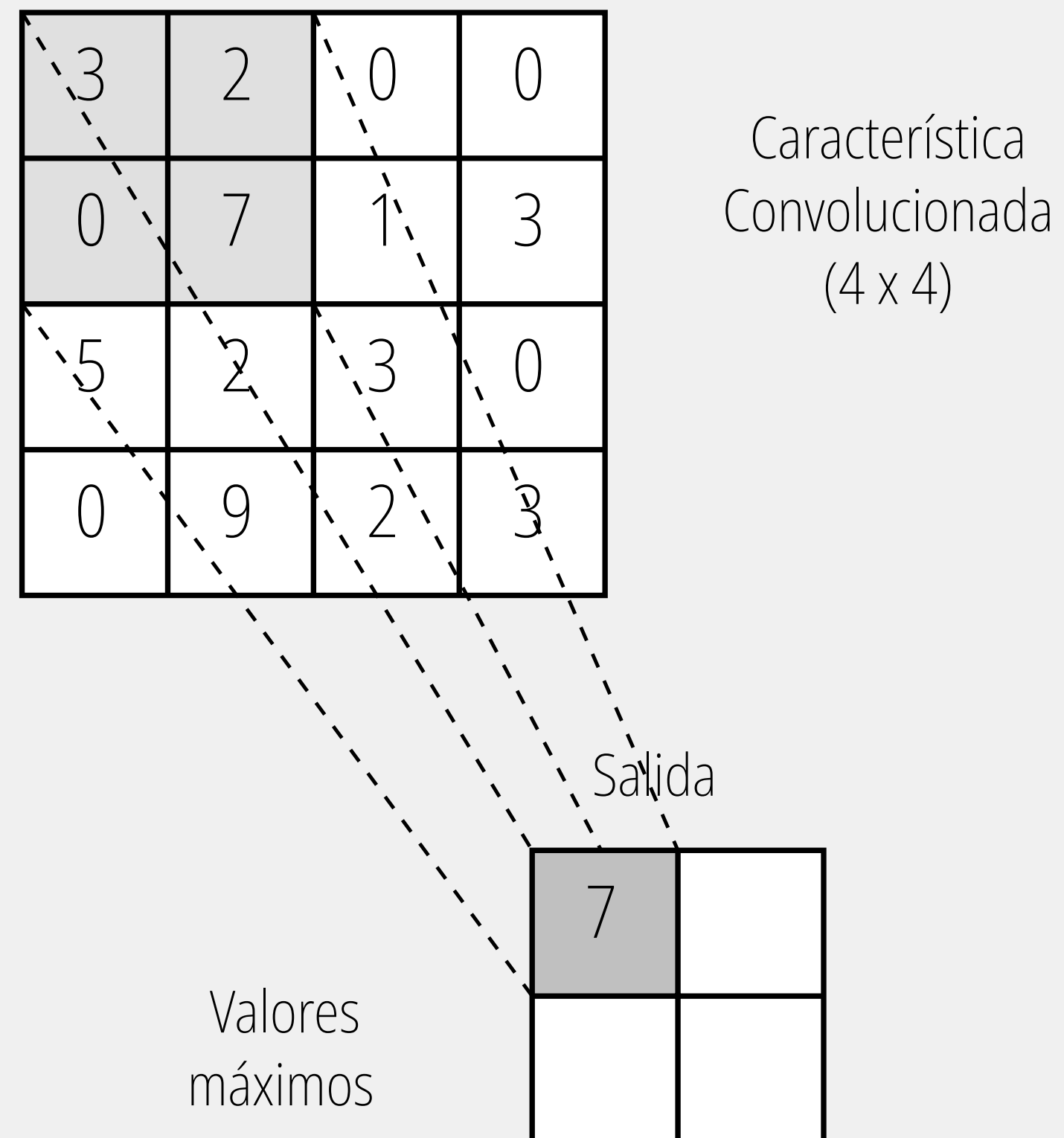
N = tamaño de la imagen

F = tamaño del filtro

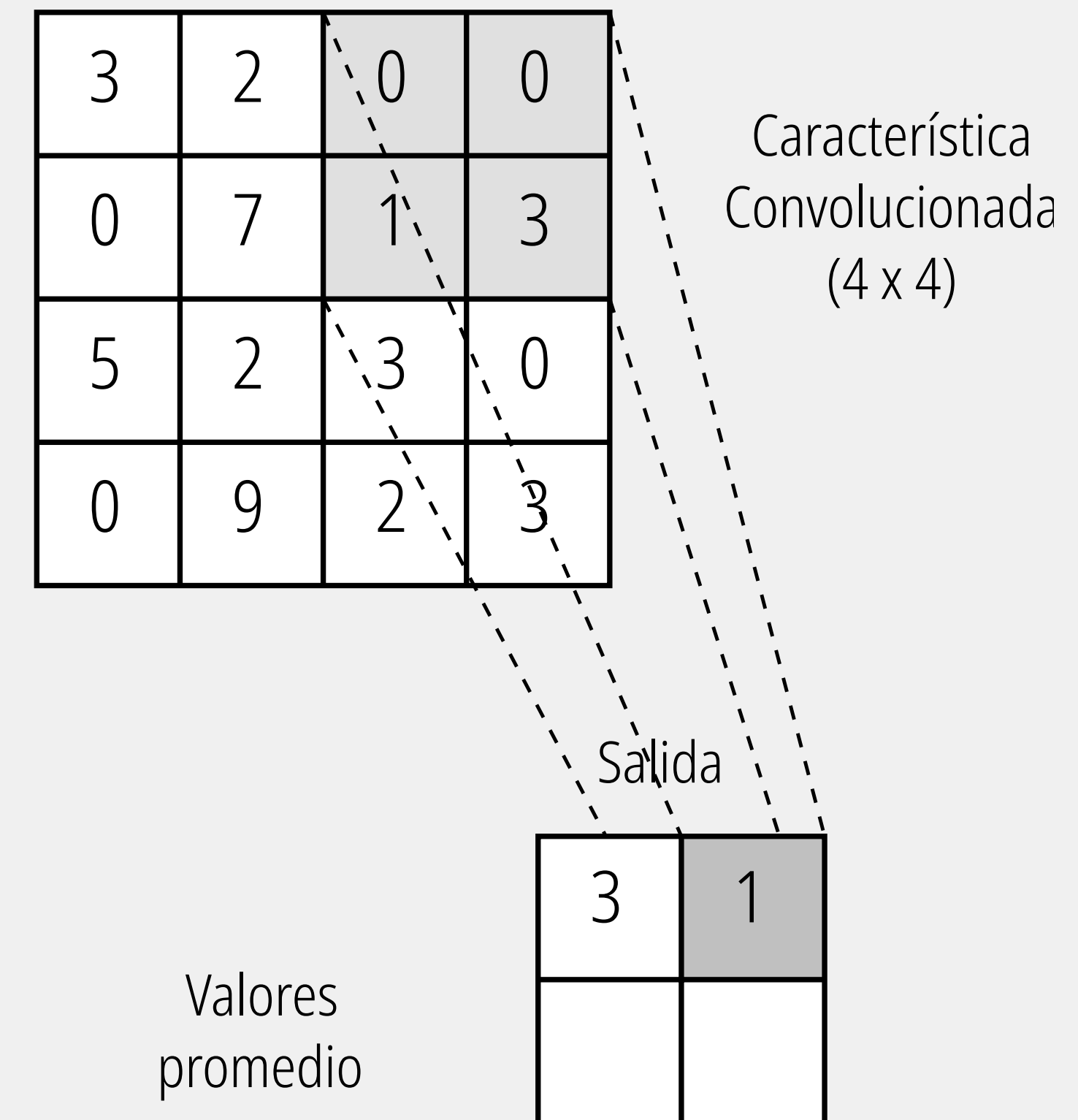
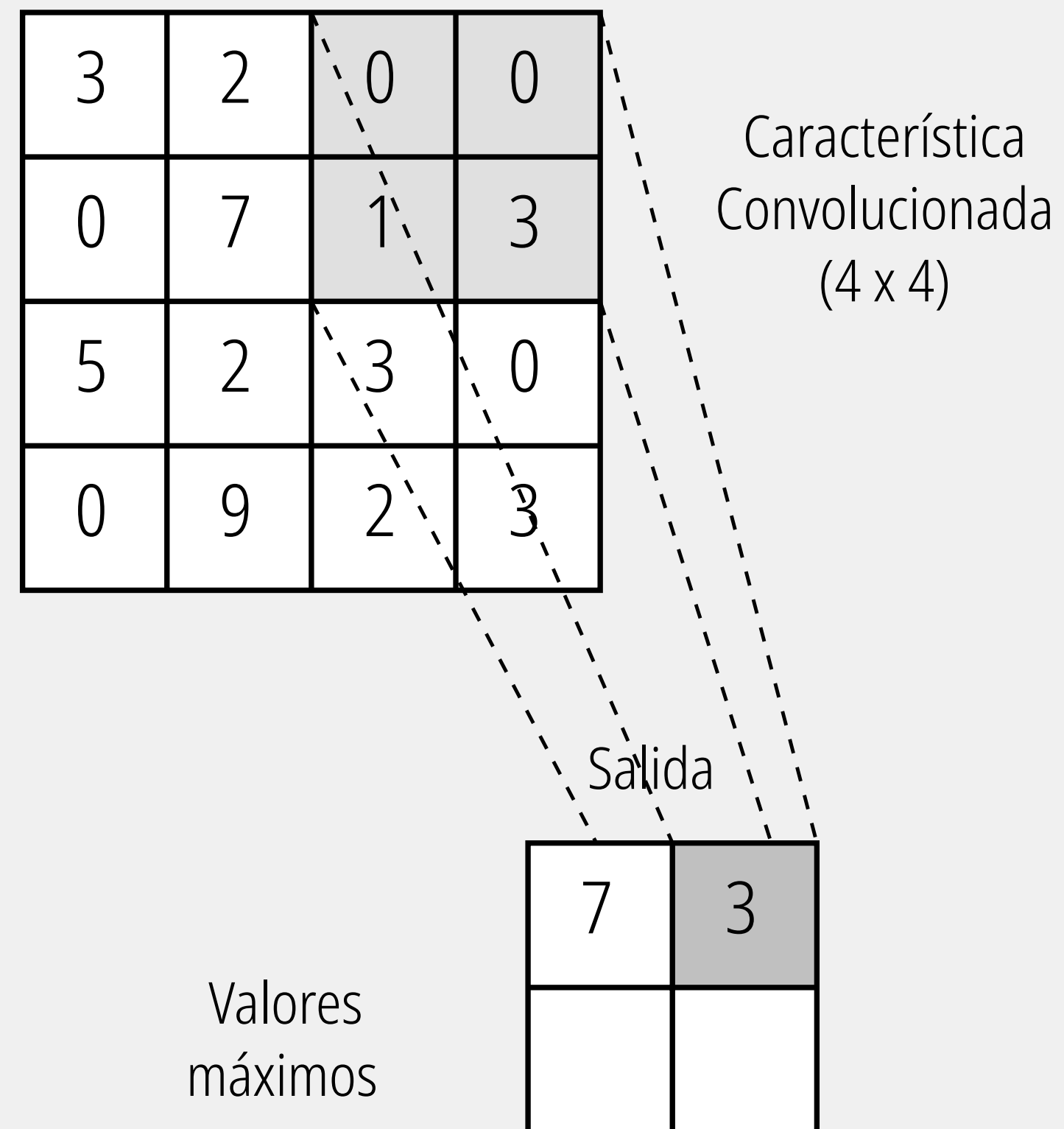
P = cantidad de relleno (padding)

S = paso (stride)

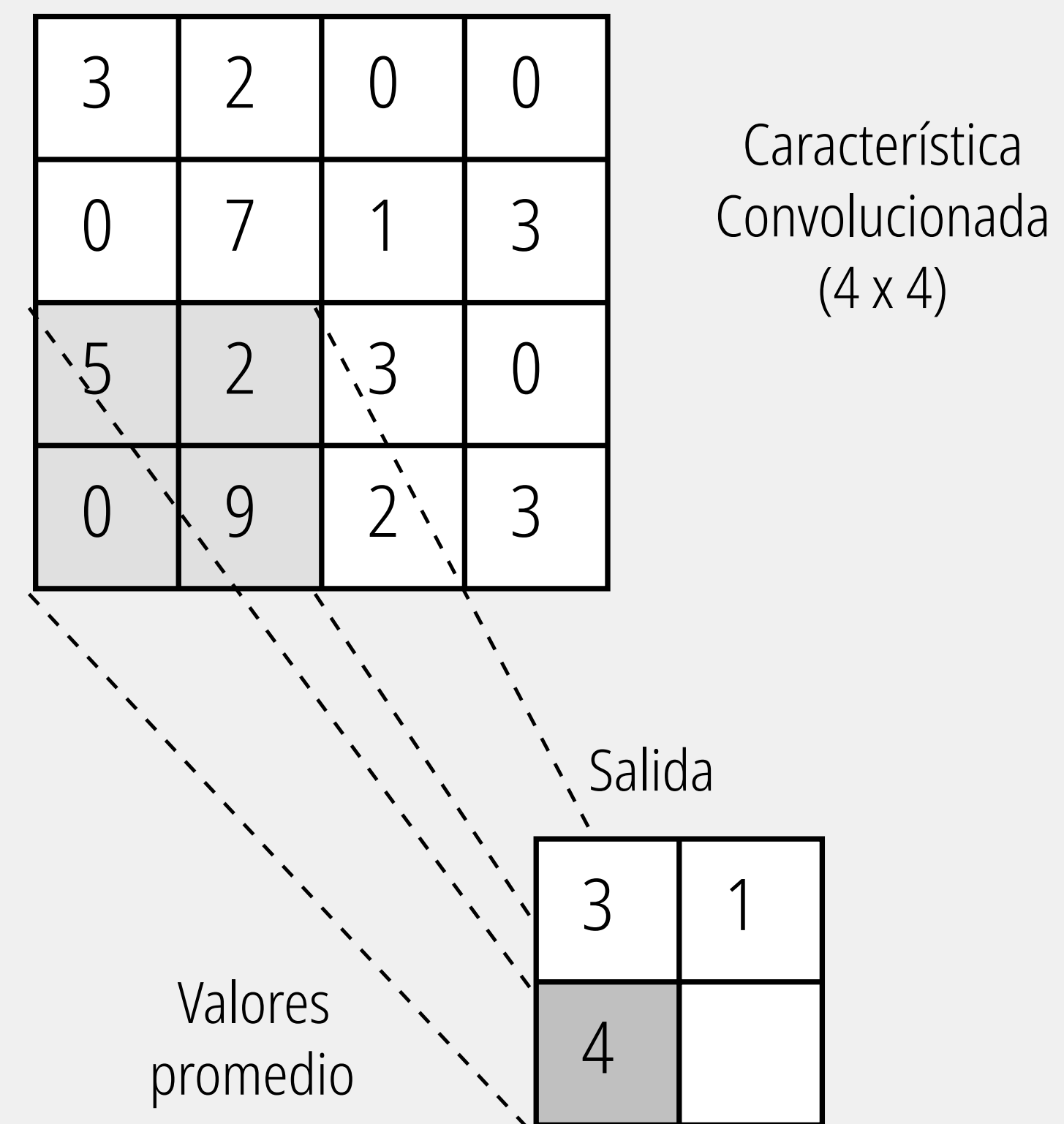
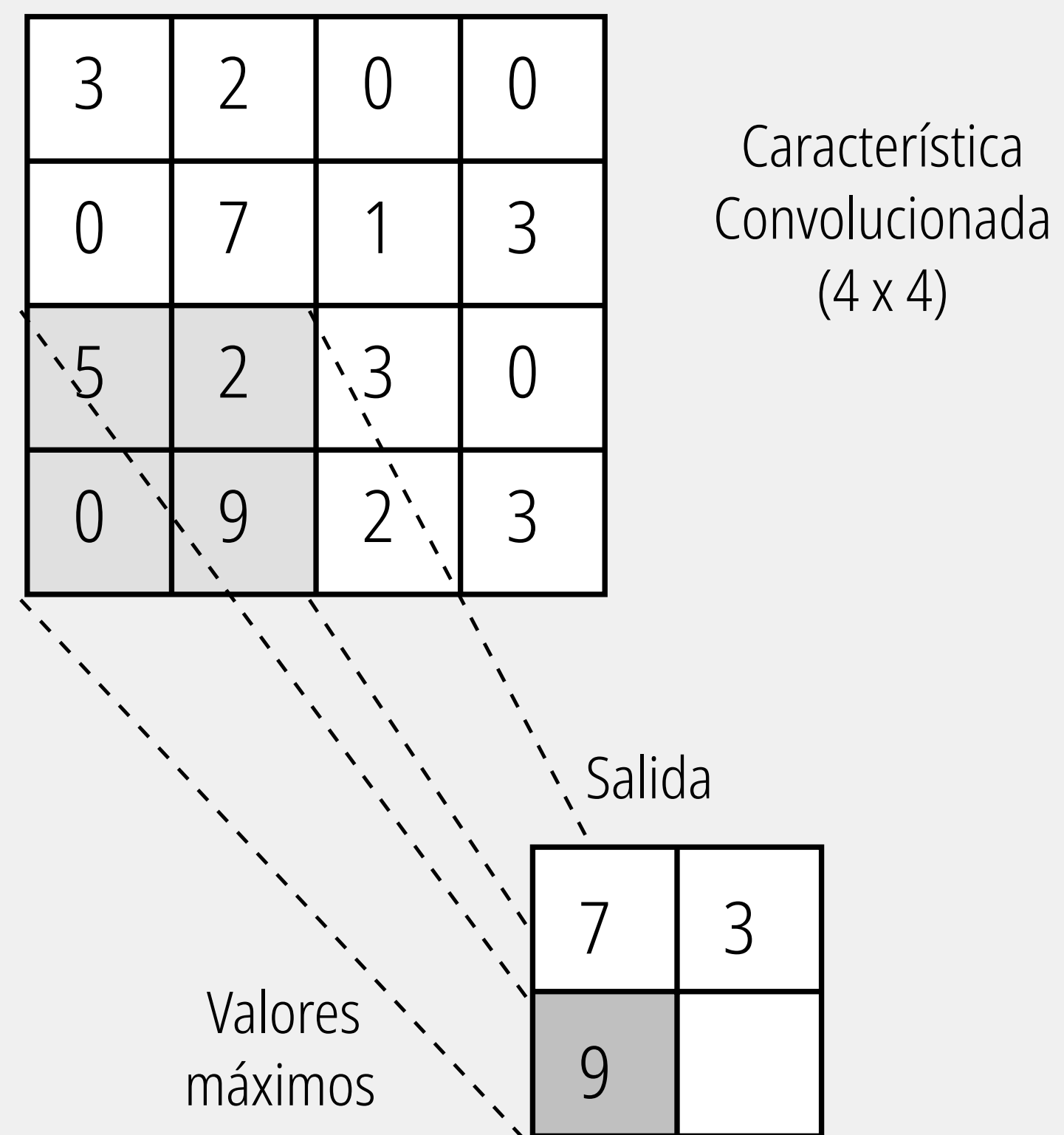
Max Pooling y Avg Pooling



Max Pooling y Avg Pooling



Max Pooling y Avg Pooling



Max Pooling y Avg Pooling

